

Problem Set C — Numerical Methods (Week + Integration)

Programming and Numerical Methods for Economics (ECNM)

The University of Edinburgh · School of Economics

Instructions. This problem set covers material from Week and integrates ideas from earlier weeks: root-finding with `scipy.optimize`, unconstrained and constrained optimisation, and applications (Solow model, consumer choice). You will need `numpy`, `scipy`, and `matplotlib`.

This version includes solutions.

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
from scipy import optimize
```

Question — Root-finding: supply and demand

Consider a market with inverse demand $P^D(Q) = 100 - 2Q$ and inverse supply $P^S(Q) = 10 + 3Q$.

(a) Solve for the equilibrium analytically: set $P^D = P^S$ and find Q^* and P^* by hand (the answer as a comment).

(b) Define an excess-demand function `excess_demand(Q)` that returns $P^D(Q) - P^S(Q)$. The equilibrium is the root of this function.

(c) Plot `excess_demand(Q)` for $Q \in [0, 30]$. Verify visually that it crosses zero near analytical answer.

(d) Use `optimize.fsolve` to find the root numerically. Compare to your analytical answer.

```
In [ ]: # --- Solution ---

# (a) Analytical: 100 - 2Q = 10 + 3Q => 90 = 5Q => Q* = 18, P* = 100

# (b)
def excess_demand(Q):
    return (100 - 2*Q) - (10 + 3*Q)

# (c)
Q_grid = np.linspace(0, 30, 200)
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(Q_grid, excess_demand(Q_grid), color='steelblue', linewidth=2)
ax.axhline(0, color='red', linestyle='--', alpha=0.5)
ax.set_xlabel('Q')
```

```

ax.set_ylabel('Excess demand')
ax.set_title('Excess demand function')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# (d)
Q_star = optimize.fsolve(excess_demand, 10)[0]
P_star = 100 - 2 * Q_star
print(f"Numerical: Q* = {Q_star:.4f}, P* = {P_star:.4f}")
print(f"Analytical: Q* = 18, P* = 64")

```

Question — Root-finding: the Solow steady state

In the Solow model, the steady-state capital per worker k^* solves:

$$s \cdot A \cdot (k^*)^\alpha - \delta \cdot k^* = 0$$

Use $A = 1$, $\alpha = 0.33$, $\delta = 0.05$, $s = 0.20$.

(a) Define a function `solow_ss(k, s, A, alpha, delta)` that returns $s \cdot A \cdot (k^*)^\alpha - \delta \cdot k$.

(b) Plot the function for $k \in [0.1, 500]$. Where does it cross zero?

(c) Use `optimize.fsolve` to find k^* . Compute the corresponding steady-state output $y^* = A \cdot (k^*)^\alpha$ and consumption $c^* = (1-s) y^*$.

(d) **Comparative statics:** Repeat for $s = 0.1, 0.2, 0.3, 0.4$. results in a dictionary and print a table showing s , k^* , y^* , and c^* . Which saving gives the highest consumption?

```

In [ ]: # --- Solution ---

A, alpha, delta = 1, 0.33, 0.05

# (a)
def solow_ss(k, s, A, alpha, delta):
    return s * A * k**alpha - delta * k

# (b)
s = 0.20
k_grid = np.linspace(0.1, 100, 500)
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(k_grid, solow_ss(k_grid, s, A, alpha, delta), color='steelblue',
ax.axhline(0, color='red', linestyle='--', alpha=0.5)
ax.set_xlabel('k (capital per worker)')
ax.set_ylabel('s*A*k^alpha - delta*k')
ax.set_title('Solow steady-state condition')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

```
# (c)
k_star = optimize.fsolve(solow_ss, 10, args=(s, A, alpha, delta))[0]
y_star = A * k_star**alpha
c_star = (1 - s) * y_star
print(f"s = {s}: k* = {k_star:.2f}, y* = {y_star:.4f}, c* = {c_star:.4f}")

# (d)
print("\nComparative statics:")
print(f"{'s':>6} {'k*':>10} {'y*':>10} {'c*':>10}")
print("-" * 40)

for s_val in [0.10, 0.20, 0.30, 0.40]:
    k = optimize.fsolve(solow_ss, 10, args=(s_val, A, alpha, delta))[0]
    y = A * k**alpha
    c = (1 - s_val) * y
    print(f"{'s_val':>6.2f} {'k':>10.2f} {'y':>10.4f} {'c':>10.4f}")

# Higher s means higher k* and y*, but c* is maximised at an
# intermediate saving rate (the Golden Rule level).
```

Question — Unconstrained optimisation: compa algorithms

Consider the **Rosenbrock function** in two dimensions: $f(x_1, x_2) = (1 - x_2)^2 + 100(x_1 - x_2^2)^2$

The global minimum is at $(x_1, x_2) = (0, 1)$ with $f = 0$.

(a) Define `rosenbrock(X)` where `X` is a 2-element array.

(b) Starting from $x_0 = (-1, 1)$, minimise using:

- BFGS (default `optimize.minimize`)
- Nelder–Mead

For each, print the solution, the function value at the solution, the number of function evaluations (`res.nfev`), and whether the algorithm reported success.

(c) Now use the steeper variant $f(x_1, x_2) = (1 - x_2)^2 + 100000(x_1 - x_2^2)^2$ (coefficient changed from 100 to 100000). Does either method struggle? Report function evaluations.

```
In [ ]: # --- Solution ---

# (a)
def rosenbrock(X):
    x1, x2 = X
    return (1 - x2)**2 + 100 * (x2 - x1**2)**2

# (b)
x0 = np.array([-1, 1])
```

```

print("=== Standard Rosenbrock (coeff = 100) ===")
for method in ['BFGS', 'Nelder-Mead']:
    res = optimize.minimize(rosenbrock, x0, method=method)
    print(f"\n{method}:")
    print(f"  Solution:  x = [{res.x[0]:.6f}, {res.x[1]:.6f}]")
    print(f"  f(x*) =    {res.fun:.2e}")
    print(f"  Func evals: {res.nfev}")
    print(f"  Success:    {res.success}")

# (c)
def rosenbrock_steep(X):
    x1, x2 = X
    return (1 - x1)**2 + 500 * (x2 - x1**2)**2

x0_hard = np.array([-2, 2])

print("\n=== Steep Rosenbrock (coeff = 500) from x0 = (-2, 2) ===")
for method in ['BFGS', 'Nelder-Mead']:
    res = optimize.minimize(rosenbrock_steep, x0_hard, method=method)
    print(f"\n{method}:")
    print(f"  Solution:  x = [{res.x[0]:.6f}, {res.x[1]:.6f}]")
    print(f"  f(x*) =    {res.fun:.2e}")
    print(f"  Func evals: {res.nfev}")
    print(f"  Success:    {res.success}")

# Nelder-Mead typically needs more function evaluations and may
# struggle with the steeper version from a distant starting point.

```

Question — Bounded optimisation: portfolio allocation

An investor allocates wealth between two assets. Asset returns are:

- Asset 1: expected return μ_1 , variance σ_1^2
- Asset 2: expected return μ_2 , variance σ_2^2
- Covariance: σ_{12}

The investor chooses a weight w in Asset 1 (with $1-w$ in Asset 2) to **minimise portfolio variance**:

$$\sigma_p^2(w) = w^2 \sigma_1^2 + (1-w)^2 \sigma_2^2 + 2w(1-w) \sigma_{12}$$

(a) Define `portfolio_var(w)` implementing the formula.

(b) Plot portfolio variance for $w \in [0, 1]$. Where is the minimum approximately?

(c) Use `optimize.minimize` with `method='L-BFGS-B'` and bounds $w \in [0, 1]$ to find the minimum-variance weight w^* . What is the portfolio's expected return at this weight?

(d) Now add a constraint: the portfolio must achieve an expected return of **at least** 6%. That is, $w\mu_1 + (1-w)\mu_2 \geq 0.06$. Use `method='SLSQP'` with the constraint. What is the new optimal w ?

```
In [ ]: # --- Solution ---

mu1, mu2 = 0.08, 0.03
s1_sq, s2_sq, s12 = 0.04, 0.01, 0.005

# (a)
def portfolio_var(w):
    return w**2 * s1_sq + (1-w)**2 * s2_sq + 2 * w * (1-w) * s12

# (b)
w_grid = np.linspace(0, 1, 200)
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(w_grid, portfolio_var(w_grid), color='steelblue', linewidth=2)
ax.set_xlabel('w (weight in Asset 1)')
ax.set_ylabel('Portfolio variance')
ax.set_title('Portfolio variance as a function of allocation')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# (c)
res = optimize.minimize(portfolio_var, 0.5, method='L-BFGS-B', bounds=[(0, 1)])
w_star = res.x[0]
exp_return = w_star * mu1 + (1 - w_star) * mu2
print(f"Minimum-variance weight: w* = {w_star:.4f}")
print(f"Portfolio variance: {res.fun:.6f}")
print(f"Portfolio std dev: {np.sqrt(res.fun):.4f}")
print(f"Expected return: {exp_return:.4f} ({100*exp_return:.2f}%)")

# (d)
def return_constraint(w):
    return w * mu1 + (1 - w) * mu2 - 0.06 # >= 0

cons = {'type': 'ineq', 'fun': return_constraint}
res2 = optimize.minimize(portfolio_var, 0.5, method='SLSQP',
                        bounds=[(0, 1)], constraints=cons)
w_constrained = res2.x[0]
exp_ret2 = w_constrained * mu1 + (1 - w_constrained) * mu2
print(f"\nConstrained (return >= 6%):")
print(f" w* = {w_constrained:.4f}")
print(f" Portfolio variance: {res2.fun:.6f}")
print(f" Expected return: {100*exp_ret2:.2f}%)")
```

Question — Root-finding meets simulation: the IS curve

Consider a simple IS-curve model where output Y satisfies:

$$Y = C(Y) + I(r) + G$$

with consumption $C(Y) = c_0 + c_1(Y - T)$, investment $I(r) = \bar{I} - d \cdot r$, government spending G , and taxes T .

Parameters: $c_0 = 100$, $c_1 = 0.6$, $\bar{I} = 200$, $d = 500$, $G = 150$, $T = 100$.

(a) Define `is_equation(Y, r)` returning $Y - C(Y) - I(r) - G$ (which equals zero in equilibrium).

(b) For a given interest rate $r = 0.05$, use `fsolve` to find equilibrium Y^* .

(c) Compute Y^* for $r \in \{0.01, 0.05, 0.11, \dots, 0.1\}$. Plot the IS curve (Y on the y-axis, r on the x-axis). Does it slope downward as theory predicts?

(d) **Fiscal multiplier:** Increase G from 150 to 200 (keeping $r = 0.05$). What is the new Y^* ? Compute $\Delta Y / \Delta G$. Compare to the theoretical Keynesian multiplier $1 / (1 - c_1)$.

```
In [ ]: # --- Solution ---

c0, c1, I_bar, d, G, T = 100, 0.6, 200, 500, 150, 100

# (a)
def is_equation(Y, r, c0=c0, c1=c1, I_bar=I_bar, d=d, G=G, T=T):
    C = c0 + c1 * (Y - T)
    I = I_bar - d * r
    return Y - C - I - G

# (b)
r = 0.05
Y_star = optimize.fsolve(is_equation, 1000, args=(r,))[0]
print(f"At r = {r}: Y* = {Y_star:.2f}")

# (c)
r_values = np.arange(0.01, 0.11, 0.01)
Y_values = []
for r_val in r_values:
    Y_eq = optimize.fsolve(is_equation, 1000, args=(r_val,))[0]
    Y_values.append(Y_eq)

fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(Y_values, r_values, 'o-', color='steelblue', markersize=6)
ax.set_xlabel('Output (Y)')
ax.set_ylabel('Interest rate (r)')
ax.set_title('IS Curve')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
# Yes, it slopes downward as theory predicts.

# (d)
G_new = 200
Y_new = optimize.fsolve(is_equation, 1000, args=(0.05, c0, c1, I_bar, d,
delta_Y = Y_new - Y_star
delta_G = G_new - G
multiplier_numerical = delta_Y / delta_G
multiplier_theory = 1 / (1 - c1)
```

```

print(f"\nFiscal multiplier:")
print(f"   Y* (G=150) = {Y_star:.2f}")
print(f"   Y* (G=200) = {Y_new:.2f}")
print(f"   dY/dG (numerical) = {multiplier_numerical:.4f}")
print(f"   1/(1-c1) (theory) = {multiplier_theory:.4f}")

```

Question — Putting it all together: the N-dimens Rosenbrock

The Rosenbrock function generalises to N dimensions:

$$f(\mathbf{x}) = \sum_{i=1}^{N-1} \left[(1 - x_i)^2 + 100(x_{i+1} - x_i^2)^2 \right]$$

In the Week 11 lecture, this was written out term by term for $N=3$. Your job is to make it

(a) Write a function `rosenbrock_nd(X)` that works for **any** length of input array `X`, using a `k` vectorised NumPy).

(b) Verify that for $N=2$ your function matches the 2D Rosenbrock from Question 3. Test points: $(0, 0)$, $(1, 1)$, $(-1, 2)$.

(c) Minimise the $N=10$ Rosenbrock using BFGS from `x0 = np.zeros(N)`. Report the function value, and number of evaluations. Is the solution close to $(1, 1, \dots, 1)$?

(d) Time BFGS vs Nelder–Mead for $N=10$ (use `%%time` or `import time`). Which is faster? Which uses fewer function evaluations?

```

In [ ]: # --- Solution ---
import time

# (a)
def rosenbrock_nd(X):
    total = 0
    for i in range(len(X) - 1):
        total += (1 - X[i])**2 + 100 * (X[i+1] - X[i]**2)**2
    return total

# (b)
test_points = [np.array([0, 0]), np.array([1, 1]), np.array([-1, 2])]
print("Verification (N=2):")
for pt in test_points:
    val_nd = rosenbrock_nd(pt)
    val_2d = rosenbrock(pt) # from Q3
    print(f"   x = {pt}:   rosenbrock_nd = {val_nd:.4f},   rosenbrock_2d = {val_2d:.4f}")

# (c)
N = 10
x0 = np.zeros(N)
res_bfgs = optimize.minimize(rosenbrock_nd, x0, method='BFGS')
print(f"\nBFGS (N={N}):")

```

```

print(f"  Solution:  {np.round(res_bfgs.x, 4)}")
print(f"  f(x*) =    {res_bfgs.fun:.2e}")
print(f"  Func evals: {res_bfgs.nfev}")
print(f"  Success:    {res_bfgs.success}")
print(f"  Close to (1,...,1)? Max deviation = {np.max(np.abs(res_bfgs.x -

# (d)
print(f"\nTiming comparison (N={N}):")

tic = time.time()
res_b = optimize.minimize(rosenbrock_nd, x0, method='BFGS')
t_bfgs = time.time() - tic

tic = time.time()
res_nm = optimize.minimize(rosenbrock_nd, x0, method='Nelder-Mead')
t_nm = time.time() - tic

print(f"  BFGS:        {t_bfgs:.4f}s, {res_b.nfev} evals, f = {res_b.fun:.
print(f"  Nelder-Mead: {t_nm:.4f}s, {res_nm.nfev} evals, f = {res_nm.fun:
# BFGS is generally faster and uses far fewer function evaluations
# because it exploits gradient information.

```

End of Problem Set C.